

Rapport de projet

*Master 1 Informatique
Projet de Génie Logiciel 2025*

***Compression de données pour accélérer
la transmission***

Réalisé par :
Cazacu Ion

Sommaire

I.	Introduction.....	3
I.1	Contexte du projet.....	3
I.2	Objectifs principaux.....	3
I.3	Langage et outils utilisés.....	4
II.	Structure du projet.....	5
III.	Détails des algorithmes.....	7
IV.1	BitPackingNoOverlap.....	7
IV.2	BitPackingOverlap.....	8
IV.3	BitPackingOverflow.....	9
IV.	Mesures et protocole expérimental.....	11
V.1	But.....	11
V.2	Méthodologie.....	11
V.	Résultats et analyse.....	14
V.1	Cas 1 : Tableau avec 1000 valeurs homogènes entre 0 et 10000.....	14
V.2	Cas 2 : Tableau de 1000 valeurs avec outliers modérés.....	15
V.3	Cas 3 : Tableau de 1000 valeurs avec outliers extrêmes.....	15
V.4	Synthèse générale.....	16
VI.	Perspectives et améliorations.....	18
VI.1	Gestion des entier négatifs.....	18
VI.2	Optimisation des performances.....	19
VI.3	Extension à d'autres types de données.....	19
VI.4	Optimisation de la zone de débordement.....	19
VI.5	Intégration dans un environnement applicatif.....	20
VII.	Conclusion.....	21

Introduction

1. Contexte du projet

À mesure que le trafic de données sur Internet ne cesse de croître, la vitesse et l'efficacité des transmissions se placent au cœur des préoccupations. Or, dans de nombreux systèmes, chaque valeur est logée sur 32 bits, même lorsqu'une représentation bien plus réduite suffirait. Le surplus d'espace gaspillé alourdit le réseau sans nécessité et rallonge les temps de transfert.

Ce projet a pour ambition de réduire la taille des données transmises tout en préservant l'intégrité de l'information, grâce à une méthode nommée Bit Packing. Cette approche consiste à n'utiliser que le nombre de bits strictement nécessaire pour représenter chaque entier. L'objectif ne se cantonne pas à la simple compression : il s'agit également de garantir un accès direct aux éléments une fois compressés, ce qui reste une exception dans les solutions classiques.

Le projet s'attache à affiner l'équilibre entre la taille des données et la vitesse de traitement. La compression doit être suffisamment puissante pour compenser le coût supplémentaire du calcul, tout en garantissant la fiabilité et la réversibilité des informations.

2. Objectifs principaux

L'objectif principal de ce projet est de concevoir, implémenter et évaluer plusieurs méthodes de compression d'entiers afin de réduire la quantité de données à transmettre tout en maintenant la possibilité d'un accès direct à chaque élément compressé. Contrairement aux méthodes de compression classiques qui nécessitent une décompression complète pour accéder à une donnée, le Bit Packing permet une lecture partielle et efficace, essentielle dans les systèmes temps réel ou à faible latence.

Pour atteindre cet objectif général, plusieurs sous-objectifs ont été définis :

- Implémenter trois variantes de l'algorithme de compression Bit Packing :
 - une version sans chevauchement (BitPackingNoOverlap),
 - une version avec chevauchement (BitPackingOverlap),
 - une version avec zone de débordement (BitPackingOverflow) pour les valeurs extrêmes.
- Mesurer précisément les performances de chaque approche à l'aide d'un protocole expérimental fiable basé sur `System.nanoTime()`.
- Comparer les temps de compression, de décompression et d'accès direct, ainsi que le ratio de compression obtenu.

- Déterminer un seuil de latence au-delà duquel la compression devient plus avantageuse que la transmission directe des données.
- Fournir une interface utilisateur interactive (via Main.java) permettant de tester, visualiser et comparer facilement les différents modes de compression.

En résumé, le projet a pour ambition de démontrer expérimentalement que la compression peut accélérer la transmission des tableaux d'entiers sans compromettre ni la précision ni la facilité d'accès aux données.

3. Langage et outils utilisés

Le projet a été intégralement développé en Java, car il offre un contrôle précis sur la gestion binaire via les opérateurs de décalage (<<, >>, >>>) et les opérations logiques (&, |, ^), indispensables à l'implémentation du Bit Packing.

L'environnement de développement utilisé est Visual Studio Code. La compilation et l'exécution ont été effectuées via la ligne de commande :

- **javac *.java** pour la compilation,
- **java Main** pour l'exécution du programme principal.

Structure du projet

Le projet a été conçu selon une architecture modulaire, favorisant la lisibilité, la réutilisabilité du code et la clarté de la logique de traitement.

L'ensemble du code source est organisé en fichiers Java distincts, chacun correspondant à un composant fonctionnel clairement identifié :

- **BitPackedArray.java :**
Structure de données représentant un tableau compressé. Elle stocke à la fois le tableau de mots compressés et les métadonnées nécessaires à la décompression (nombre d'éléments n , taille des blocs k , informations de débordement, etc.). Cette classe sert d'intermédiaire entre les algorithmes de compression et les opérations de lecture/décompression.
- **BitPacking.java :**
Interface principale définissant le contrat que doivent respecter toutes les méthodes Bitpacking. Elle impose l'implémentation de trois fonctions essentielles : `compress()`, `decompress()`, et `get()`. Ce choix garantit une uniformité d'utilisation entre les différentes stratégies.
- **BitPackingNoOverlap.java :**
Première implémentation de l'interface BitPacking. Elle réalise une compression simple sans chevauchement de bits : les entiers compressés ne peuvent pas déborder sur 2 entiers consécutifs.
- **BitPackingOverlap.java :**
Variante permettant le chevauchement entre entiers. Les entiers peuvent s'étendre sur deux entiers consécutifs afin d'optimiser l'espace mémoire. Cette version augmente la densité de compression.
- **BitPackingOverflow.java :**
Implémentation avancée intégrant une zone de débordement (overflow). Lorsque certaines valeurs nécessitent plus de bits que la moyenne, elles sont stockées dans une zone séparée. Le tableau principal contient alors un indicateur (0 = valeur normale, 1 = référence vers la zone de débordement). Cette approche réduit la taille moyenne tout en préservant l'accès direct.
- **BitPackingFactory.java :**
Implémente le Design Pattern Factory permettant de créer dynamiquement une instance du type de BitPacking souhaité à partir d'une chaîne de caractères ("nooverlap", "overlap", "overflow"). Ce mécanisme facilite les tests et le changement de stratégie à l'exécution sans modifier le code principal.
- **BitPackingUtils.java :**
Contient des fonctions utilitaires pour vérifier la consistance des tableaux avant et après compression pour éviter les mauvaises entrées.

- **Main.java :**

Point d'entrée du programme. Il gère l'interface utilisateur en console, le menu interactif, la sélection des algorithmes, la génération des tableaux, et le lancement des benchmarks. Le Main permet à l'utilisateur de manipuler directement les différentes stratégies de compression et d'observer leurs performances.

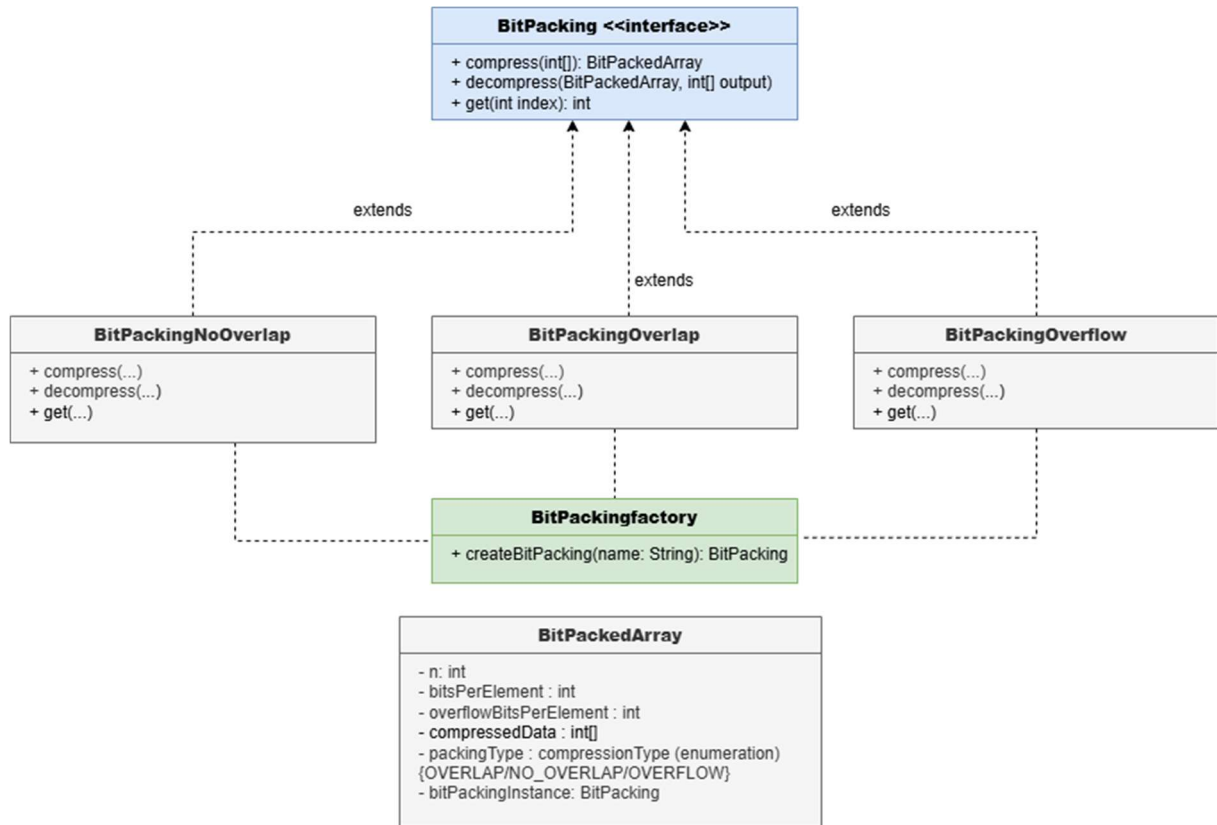


Figure 1 : schéma simplifié des classes principales

Détails des algorithmes

1. BitPackingNoOverlap

Le concept du Bit Packing repose sur la constatation suivante : si tous les entiers d'un tableau peuvent être représentés avec k bits au lieu des 32 habituels, il est possible de regrouper plusieurs de ces entiers dans une série de mots binaires de 32 bits.

Dans la version NoOverlap, chaque entier compressé reste entièrement contenu dans un seul entier, sans empiéter sur le suivant.

Par exemple, si $k = 10$ et que chaque mot contient 32 bits, on peut stocker :

- 3 entiers complets dans le premier mot ($3 \times 10 = 30$ bits),
- et laisser 2 bits inutilisés à la fin du mot.

Aucun entier ne déborde sur le mot suivant, d'où le nom "NoOverlap".

Étapes principales de la compression :

1. Détermination du nombre minimal de bits nécessaires (k) :

Le programme identifie le plus grand entier du tableau et calcule le nombre minimal de bits nécessaires pour le représenter :

$$k = \lceil \log_2 (\max + 1) \rceil.$$

2. Allocation du tableau compressé

Le nombre d'entiers nécessaires (mots) est calculé à partir de la taille du tableau d'origine n et de k : $\text{mots} = \text{ceil} (n / (32 / k))$.

Un tableau d'entiers de cette taille est ensuite créé pour stocker les valeurs compressées.

3. Écriture séquentielle des entiers

Chaque entier est écrit successivement dans son bloc réservé à l'intérieur du mot de 32 bits. L'algorithme utilise un masque binaire $((1 \ll k) - 1)$ pour isoler les k bits significatifs de chaque valeur avant de les insérer à la bonne position.

4. Aucun chevauchement entre les mots

Si un mot ne dispose pas de suffisamment d'espace pour stocker le prochain entier, le programme passe au mot suivant sans tenter de le découper.

Cela simplifie les opérations binaires et améliore la clarté, au prix d'un léger gaspillage d'espace (quelques bits non utilisés).

Étapes de la décompression :

La décompression est l'inverse exacte de la compression :

1. Pour chaque entier à restaurer, on identifie le mot correspondant et la position de départ du bloc de k bits.
2. On extrait les bits nécessaires par décalage à droite ($\gg$) et masquage ($\& \text{mask}$).
3. Les valeurs obtenues sont remplacées dans le tableau d'origine.

L'opération est simple et symétrique, assurant une réversibilité parfaite entre compression et décompression.

Accès direct avec la fonction `get(i)` :

La fonction `get(i)` permet d'accéder directement au i^{e} élément compressé sans décompresser l'ensemble du tableau.

L'index de départ du mot est obtenu par :

```
int elementsPerWord = 32 / k;  
int wordIndex = index / elementsPerWord;  
int offset = (index % elementsPerWord) * k;
```

Puis, on extrait la valeur par masquage et décalage.

Comme chaque entier tient dans un seul mot, la récupération est immédiate et ne nécessite aucune recomposition sur plusieurs mots.

2. BitPackingOverlap

L'algorithme BitPackingOverlap reprend les fondements du Bit Packing classique, mais introduit une amélioration essentielle : la possibilité pour un entier de déborder partiellement sur deux mots consécutifs. Cette approche permet d'optimiser l'utilisation de l'espace mémoire, en supprimant les bits inutilisés laissés à la fin des mots dans la version NoOverlap.

Principe général :

Contrairement à la méthode précédente, BitPackingOverlap ne s'interdit pas qu'un entier chevauche deux blocs de 32 bits. Ainsi, si un entier compressé ne tient pas entièrement dans l'entier courant, les bits restants sont écrits dans l'entier suivant. Cela assure une densité maximale des données compressées : aucun bit n'est perdu entre deux entiers successifs.

Exemple : Si $k = 12$, on peut stocker dans un mot de 32 bits :

- les deux premiers entiers compressés entièrement (24 bits),
- et les 8 premiers bits du troisième entier compressé (les 4 bits restants iront dans l'entier suivant).

Calcul du nombre de bits nécessaires (k) :

Elle est identique à la version précédente : $k = \lceil \log_2(\max + 1) \rceil$.

Écriture séquentielle avec chevauchement contrôlé :

Chaque entier est inséré bit à bit au bon endroit dans le flux binaire.

Si la position de fin dépasse 32 bits, la partie excédentaire est copiée dans le mot suivant.

Lecture symétrique (décompression) :

Pour reconstituer un entier, l'algorithme lit les bits nécessaires depuis un ou deux mots.

Les fragments sont ensuite recombinaés par d calages et op rations logiques.

Acc s direct `get(i)` :

Plus complexe que dans la version `NoOverlap`, car un entier peut  tre r parti sur deux mots. L'algorithme r cup re alors les deux segments n cessaires et les fusionne avant de renvoyer la valeur.

3. BitPackingOverflow

La m thode `BitPackingOverflow` constitue la version la plus avanc e du projet. Elle vise   r soudre une faiblesse commune aux approches pr c dentes (`NoOverlap` et `Overlap`) : toutes les valeurs sont encod es sur le m me nombre de bits k , d termin  par la plus grande valeur du tableau. Lorsque la majorit  des entiers est petite et que quelques valeurs isol es sont tr s grandes, cette uniformit  entra ne un gaspillage important d'espace.

L'id e du `BitPackingOverflow` est donc d'introduire une zone de d bordement (`overflow area`) o  seules les valeurs exceptionnelles, n cessitant plus de bits que la moyenne, sont stock es.

Principe g n ral :

Le tableau compress  est divis  en deux parties :

1. **Zone principale** : contient la majorit  des entiers, compress s sur k' bits (plus petits que k).
2. **Zone de d bordement** : contient uniquement les valeurs "trop grandes", compress es elles sur k bits.

Chaque entier de la zone principale commence par un bit de contr le :

- 0 $\rightarrow$ l'entier est cod  normalement dans la zone principale.
- 1 $\rightarrow$ l'entier est une r f rence vers la zone de d bordement.

Exemple : Pour le tableau [1, 2, 3, 1024, 4, 5, 2048], 1, 2, 3, 4, 5 sont stock es normalement, tandis que 1024 et 2048 sont plac es en fin de structure dans la zone de d bordement.

 tapes principales de la compression :

1. Analyse du tableau d'origine

Le programme d termine deux valeurs :

- o k' = nombre de bits n cessaires pour repr senter la majorit  des valeurs dans la zone principale
- o k = nombre de bits pour la plus grande valeur du tableau et sur lesquels on va compresser les entiers dans la zone `overflow`.

2. Cr ation de la zone principale

Chaque entier est examin  :

- o S'il est inf rieur au seuil $2^{(k')}$, il est cod  sur $k'+1$ bits (0 + valeur).
- o Sinon, il re oit un identifiant 1 + index vers la zone de d bordement.

3. **Écriture de la zone de débordement**

Les valeurs exceptionnelles sont enregistrées à la fin du tableau compressé, codées sur k bits chacune.

Décompression :

La décompression lit chaque entrée :

- Si le bit de tête est 0, la valeur est reconstituée directement à partir de la zone principale ;
- Si le bit de tête est 1, l'index associé permet de récupérer la valeur depuis la zone de débordement.

Ce mécanisme garantit une réversibilité parfaite tout en optimisant l'espace.

Mesures et protocole expérimental

1. But

Le but du protocole expérimental est de quantifier objectivement les performances des différentes méthodes de compression implémentées (BitPackingNoOverlap, BitPackingOverlap, BitPackingOverflow) afin d'évaluer leur efficacité réelle en termes de temps d'exécution, de taux de compression, et de gain de transmission.

L'objectif n'est pas uniquement de mesurer la vitesse ou la taille compressée, mais de déterminer expérimentalement dans quelles conditions la compression devient réellement avantageuse par rapport à une transmission non compressée.

En d'autres termes, il s'agit de mesurer à partir de quelle latence réseau la compression compense le surcoût de calcul qu'elle induit.

Le protocole doit donc répondre à trois questions fondamentales :

1. Quel est **le coût en temps CPU** pour compresser et décompresser un tableau d'entiers ?
2. Quel est **le gain en taille mémoire** (ratio de compression) obtenu pour chaque méthode ?
3. À partir de **quelle latence de transmission la compression devient plus rentable** que l'envoi direct des données non compressées ?

Ces mesures sont réalisées de manière rigoureuse et répétée, à l'aide d'un protocole standardisé reposant sur :

- Des tableaux de test de tailles et distributions variées (aléatoire uniforme, valeurs extrêmes, etc.),
- L'utilisation de la fonction `System.nanoTime()` pour obtenir des temps précis à la nanoseconde,
- La médiane de plusieurs itérations pour limiter les variations liées à l'environnement d'exécution,
- Et le calcul systématique de la taille compressée effective en bits.

Ce protocole permet ainsi d'évaluer non seulement les performances brutes (vitesse, taille), mais aussi la rentabilité pratique de la compression dans un contexte de transmission réelle.

2. Méthodologie

Afin de garantir la fiabilité et la reproductibilité des résultats, un protocole de mesure précis a été mis en place. Toutes les expériences ont été réalisées depuis le programme principal (Main.java), qui intègre un système de benchmark automatique basé sur des chronométrages à haute résolution. Chaque série de mesures repose sur plusieurs exécutions successives afin de réduire les fluctuations dues au système d'exploitation ou au garbage collector de la JVM.

Environnement de mesure :

Les tests ont été réalisés dans les conditions suivantes :

- **Langage** : Java 21
- **Chronométrage** : `System.nanoTime()`
- **Nombre d'itérations par test** : variable selon la taille du tableau (généralement 100 à 1 000 exécutions)
- **Type de données testées** : tableaux d'entiers (`int[]`) de tailles différentes (de quelques dizaines à plusieurs milliers d'éléments)
- **Distribution des valeurs** :
 - Uniforme : valeurs aléatoires comprises dans une borne maximale fixée.
 - Avec "outliers" : valeurs majoritairement faibles avec quelques entiers très élevés, simulant des cas réels hétérogènes.

Avant chaque série de mesures, la JVM est "réchauffée" (warm-up) pour réduire l'influence du chargement initial du bytecode sur les temps mesurés.

Chronométrage des fonctions :

Trois fonctions principales sont mesurées pour chaque algorithme :

a) Temps de compression

Le programme enregistre le temps exact entre le début et la fin de la fonction `compress(int[])`.

Ce temps inclut :

- le calcul du nombre de bits k nécessaires,
- la création du tableau compressé,
- et l'écriture effective des valeurs dans les mots binaires.

Formule :

$$t_{\text{compression}} = (t_{\text{fin}} - t_{\text{début}})$$

b) Temps de décompression

Même principe appliqué à la fonction `decompress(...)`. Le tableau compressé est reconverti en un tableau d'entiers, puis comparé à l'original pour garantir la réversibilité.

c) Temps d'accès direct

La fonction `get(int i)` est chronométrée séparément afin d'évaluer la rapidité d'accès à un élément compressé sans décompression complète. Plusieurs indices aléatoires sont testés pour obtenir une médiane représentative (plutôt qu'une moyenne, afin d'éliminer les valeurs extrêmes). Un checksum est calculé pour éviter que le compilateur JIT ne supprime l'appel à `get()` lors de l'optimisation.

Calcul du ratio de compression :

Pour chaque méthode, le ratio de compression est défini comme :

$$\text{Ratio} = (\text{taille_non_compressée} / \text{taille_compressée})$$

Où :

- $\text{taille_non_compressée} = n * 32 \text{ bits}$ (taille d'un tableau normal d'entiers),
- $\text{taille_compressée} = n * k_{\text{effectif}} \text{ bits}$ (ou plus dans le cas Overflow, si présence d'une zone supplémentaire).

Un ratio supérieur à 1 indique une compression effective.

Médiane et stabilité des résultats :

Chaque mesure est répétée plusieurs fois (souvent plusieurs centaines d'itérations).

Le programme calcule ensuite :

- La médiane du temps obtenu,
- L'écart-type pour évaluer la stabilité du résultat.

Les valeurs extrêmes (outliers temporels) sont ignorées pour éviter les biais dus à la JVM ou aux interruptions système. Cette approche garantit que les valeurs rapportées sont stables et représentatives.

Calcul du temps total de transmission :

Enfin, pour chaque méthode, on calcule le temps global de transmission simulé :

$T_{\text{total}} = T_{\text{compression}} + T_{\text{transmission}}$

Avec : $T_{\text{transmission}} = (\text{taille_compressée} / \text{bande_passante}) + \text{latence}$

Cette équation permet de simuler différents scénarios réseau (faible ou forte latence) et d'identifier la valeur seuil de latence t à partir de laquelle la compression devient plus avantageuse que la transmission directe.

Validation des résultats :

Après chaque test :

- Les tableaux décompressés sont comparés aux originaux à l'aide de `Arrays.equals()`,
- Toute incohérence entraîne un message d'erreur (en rouge dans la console).
Aucune différence n'a été observée dans les tests finaux, confirmant la fiabilité des algorithmes.

Résultats et analyse

L'évaluation expérimentale vise à comparer objectivement les performances des trois méthodes de compression implémentées : BitPackingNoOverlap, BitPackingOverlap et BitPackingOverflow. Les mesures ont été réalisées sur plusieurs types de tableaux, afin de mettre en évidence le comportement des algorithmes dans différents contextes de données.

Les résultats présentés ci-dessous proviennent des tests effectués avec un protocole identique pour toutes les méthodes : 100 opérations get() par test, chronométrage avec System.nanoTime(), et calcul des médianes sur plusieurs itérations pour éliminer les fluctuations de la JVM.

1. Cas 1 : Tableau avec 1000 valeurs homogènes entre 0 et 10000

Méthode	Temps de compression (ms)	Temps de décompression (ms)	Temps get (ns)	Ratio	T_threshold (ms)
NoOverlap	27.200	31.200	14.50	2.00x	> 0.058
Overlap	78.800	65.500	62.00	2.28x	> 0.112
Overflow	130.350	82.350	20.00	2.13x	> 0.188

Analyse :

Dans ce scénario, les valeurs sont toutes du même ordre de grandeur (aucune valeur extrême). Les trois méthodes obtiennent des ratios de compression similaires, compris entre 2.0x et 2.3x. La méthode Overlap tire parti du chevauchement des mots binaires pour légèrement améliorer la densité du codage (2.28x contre 2.00x pour NoOverlap), mais au prix d'un coût CPU nettement supérieur : son temps de compression est presque 3 fois plus élevé.

La méthode Overflow, quant à elle, n'apporte pas d'avantage dans ce cas homogène : l'analyse des débordements ne sert à rien, et alourdit inutilement la compression.

Son T_threshold (temps de latence à partir duquel la compression devient rentable) est le plus élevé, signe que son surcoût n'est pas compensé par un gain en taille.

En revanche, NoOverlap reste la méthode la plus équilibrée ici : rapide à exécuter, simple, et rentable dès de faibles latences réseau.

Conclusion partielle :

Pour des données homogènes, BitPackingOverlap obtient le meilleur taux de compression, mais BitPackingNoOverlap reste la plus rentable globalement grâce à sa rapidité d'exécution. La méthode Overflow n'apporte aucun gain dans ce contexte.

2. Cas 2 : Tableau de 1000 valeurs avec outliers modérés (1 valeur sur 10)

Valeurs normales $\leq 1\,000$, outlier $\leq 10\,000$

Méthode	Temps de compression (ms)	Temps de décompression (ms)	Temps get (ns)	Ratio	T_threshold (ms)
NoOverlap	3.050	2.600	8.00	2.00x	> 0.006
Overlap	3.900	3.600	13.00	2.28x	> 0.006
Overflow	11.700	4.800	18.00	2.60x	> 0.010

Analyse :

L'introduction de valeurs extrêmes commence à faire apparaître les limites des approches sans gestion de débordement. Les méthodes NoOverlap et Overlap doivent augmenter la valeur k pour pouvoir représenter les entiers les plus grands (10 000), ce qui diminue leur efficacité globale. Leur ratio reste stable, mais plafonne autour de 2.0x–2.3x.

À l'inverse, Overflow conserve un ratio supérieur (2.60x), car elle stocke la majorité des entiers sur un petit nombre de bits k' et isole uniquement les valeurs anormales dans une zone de débordement. Ce mécanisme évite le gaspillage de bits pour les petites valeurs. Le coût en temps de compression reste plus élevé (≈ 12 ms), mais la méthode devient plus rentable dès que la latence dépasse 0.01 ms, soit un seuil comparable aux autres méthodes.

Conclusion partielle :

En présence d'un faible taux de valeurs extrêmes, BitPackingOverflow commence à montrer un avantage en ratio, tout en maintenant une décompression stable et un accès direct comparable aux autres méthodes.

3. Cas 3 : Tableau de 1000 valeurs avec outliers extrêmes (1 valeur sur 50)

Valeurs normales $\leq 10\,000$, outlier $\leq 1\,000\,000\,000$

Méthode	Temps de compression (ms)	Temps de décompression (ms)	Temps get (ns)	Ratio	T_threshold (ms)
NoOverlap	1.700	2.600	5.00	1.00x	> Infinity
Overlap	5.100	1.800	12.00	1.07x	> 0.104
Overflow	16.700	1.800	11.00	2.05x	> 0.018

Analyse :

Dans ce scénario, les valeurs du tableau sont majoritairement comprises entre 0 et 10 000, mais une valeur sur cinquante atteint 1 000 000 000. Ces valeurs extrêmes forcent les algorithmes classiques (NoOverlap et Overlap) à utiliser un nombre de bits k

suffisant pour représenter la plus grande valeur du tableau (environ 30 bits). Ce choix impose une taille fixe très élevée pour tous les éléments, même ceux qui auraient pu être représentés sur une dizaine de bits.

Conséquence : la compression devient **inefficace**, voire inexistante.

Le ratio pour NoOverlap retombe à 1.00x, ce qui signifie qu'aucune réduction de taille n'a lieu, la méthode est même totalement inutile dans ce cas ($T_threshold = \infty$).

La méthode Overlap ne fait guère mieux : elle optimise un peu l'espace grâce au chevauchement, mais reste contrainte par le k maximal imposé par l'outlier.

En revanche, la méthode Overflow conserve une compression **significative** avec un ratio de 2.05x. Elle parvient à séparer efficacement les valeurs extrêmes (stockées dans une zone de débordement spécifique) des valeurs normales (compressées sur un petit $k' \approx 14$ bits). Ce découplage permet d'obtenir un gain global de taille, sans sacrifier la réversibilité ni l'accès direct. Le coût de compression reste plus élevé (≈ 16 ms), mais la méthode devient rentable dès une latence supérieure à 0.018 ms, soit bien avant les autres.

Le temps d'accès `get()` reste du même ordre de grandeur que les deux autres méthodes, confirmant que l'ajout d'une zone de débordement n'impacte pas la lecture.

Conclusion partielle :

L'algorithme BitPackingOverflow démontre ici tout son intérêt. En présence d'outliers rares mais extrêmement grands, il reste le seul à maintenir une compression efficace. Les méthodes NoOverlap et Overlap, limitées par la valeur maximale, perdent toute efficacité car elles doivent réserver un nombre excessif de bits à tous les éléments.

4. Synthèse générale :

Méthode	Points forts	Points faibles	Cas d'usage idéal
NoOverlap	Rapidité, simplicité, accès direct instantané	Ratio limité, sensible aux outliers	Données homogènes, faible latence réseau
Overlap	Bon compromis entre compacité et coût CPU	Accès direct plus lent, complexité accrue	Données d'amplitude moyenne, taille importante
Overflow	Excellent ratio sur les données hétérogènes	Coût CPU plus élevé, implémentation plus complexe	Données irrégulières, latence élevée ou bande passante limitée

Ces résultats démontrent que le choix de la méthode de compression dépend du contexte d'utilisation :

- Sur des données homogènes, la simplicité de NoOverlap suffit.
- Sur des données légèrement dispersées, Overlap offre un bon compromis.

- Et pour des données très hétérogènes contenant des valeurs extrêmes, Overflow s'impose comme la seule solution réellement efficace.

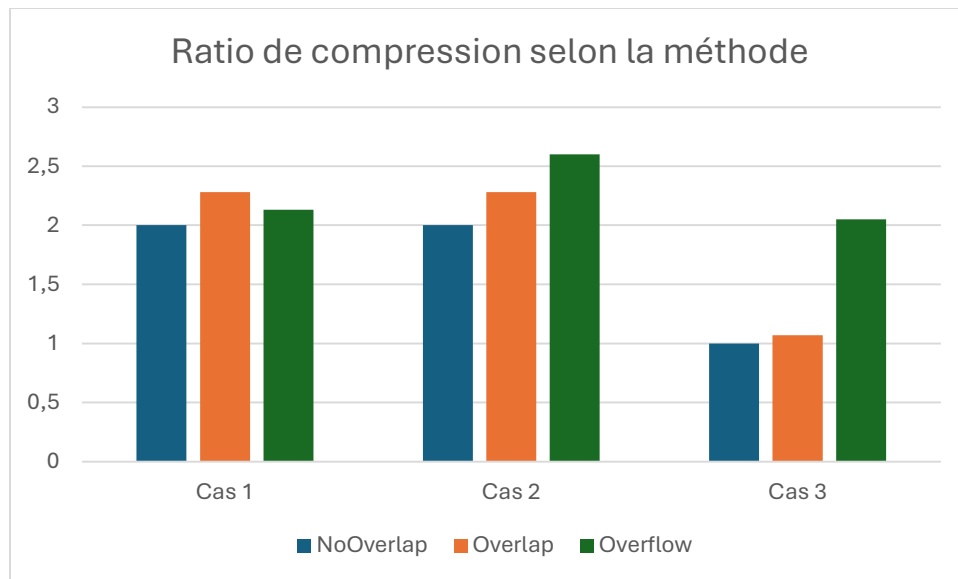


Figure 2 : graphique du ratio de compression selon la méthode

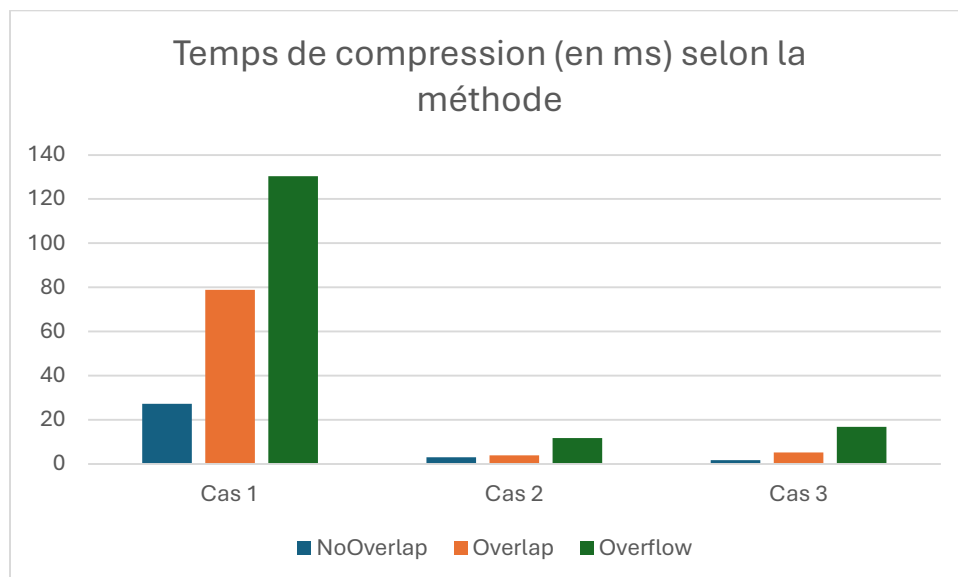


Figure 3 : graphique du temps de compression (en ms) selon la méthode

Perspectives et améliorations

Le projet présenté constitue une première implémentation complète et fonctionnelle de la compression d'entiers par Bit Packing. Cependant, plusieurs pistes d'amélioration peuvent être envisagées, tant au niveau algorithmique que fonctionnel. Ces évolutions viseraient à élargir le champ d'application du projet et à optimiser encore ses performances dans des contextes variés.

1. Gestion des entiers négatifs :

L'une des principales limites du projet actuel est qu'il ne gère que des entiers positifs. Or, dans de nombreux cas d'usage (statistiques, coordonnées, signaux, données économiques, etc.), les tableaux d'entiers peuvent contenir des valeurs négatives.

Le Bit Packing tel qu'il est actuellement conçu ne peut pas directement représenter des entiers signés, car il encode les valeurs en binaire **non signé**. Ainsi, une valeur négative comme -5 serait interprétée comme un entier positif très grand (par exemple $2^{32} - 5$).

Solutions possibles :

1. Encodage ZigZag (utilisé dans Protocol Buffers de Google) :

Cet encodage mappe les entiers signés sur des valeurs non signés en mappant les entiers positifs sur les entiers positifs pairs et ceux négatifs sur les entiers positifs impairs.

Formule : $\text{encoded} = (n < 0) \oplus (n \gg 31)$

Ainsi : $0 \rightarrow 0$; $-1 \rightarrow 1$; $1 \rightarrow 2$; $-2 \rightarrow 3$; etc.

Avantage : les nombres proches de zéro restent compressibles efficacement.

Inconvénient : nécessite une étape de transformation supplémentaire.

2. Réserve d'un bit de signe

On peut réserver un bit pour représenter le signe (0 = positif, 1 = négatif) et compresser la valeur absolue sur les $k - 1$ bits restants.

Avantage : implémentation simple.

Inconvénient : perte d'un bit de précision (ce qui réduit légèrement le ratio global).

3. Décalage de plage (offset) :

Si la plage des valeurs est connue (par exemple entre -1000 et +1000), on peut ajouter un offset à toutes les valeurs pour les rendre positives (ex. +1000).

Avantage : compression simple, aucune modification structurelle du code.

Inconvénient : nécessite une connaissance préalable des bornes de la distribution.

Ces approches permettraient de généraliser le projet à des jeux de données mixtes, tout en maintenant la compatibilité avec les algorithmes déjà implémentés (NoOverlap, Overlap, Overflow).

2. Optimisation des performances :

Plusieurs optimisations techniques pourraient améliorer la rapidité d'exécution et la consommation mémoire :

- **Vectorisation ou traitement par lots**
En traitant plusieurs entiers simultanément à l'aide d'opérations binaires groupées, on pourrait réduire considérablement le temps de compression et de décompression.
- **Utilisation de structures de flux binaires (BitSet, ByteBuffer)**
Cela permettrait de mieux contrôler la lecture et l'écriture bit-à-bit, notamment pour la méthode Overlap, où les accès aux mots peuvent se chevaucher.
- **Parallélisation du traitement**
La compression pourrait être divisée en segments indépendants, traités sur plusieurs cœurs, avant d'être fusionnés. Ce découpage permettrait d'accélérer le traitement de très grands tableaux sans perte de cohérence.

3. Extension à d'autres types de données :

Le Bit Packing utilisé ici est adapté aux entiers non signés. Cependant, la même logique pourrait être appliquée à d'autres types :

- **Flottants** : en convertissant leur mantisse/exposant en représentation fixe, pour des données scientifiques.
- **Données booléennes ou catégorielles** : qui peuvent être compressées sur 1 à 3 bits seulement.
- **Chaînes courtes** : en encodant leurs caractères selon des alphabets restreints.

Ces extensions ouvriraient la voie à un framework générique de compression pour différents types de données numériques.

4. Optimisation de la zone de débordement :

La méthode Overflow pourrait être encore améliorée :

- En triant ou regroupant les valeurs débordantes pour mieux exploiter la localité mémoire.
- En adaptant dynamiquement la taille du bit indicateur ou en utilisant plusieurs seuils (multi-level overflow) selon la distribution des valeurs.
- En ajoutant une estimation automatique du seuil optimal k' en fonction du ratio de valeurs extrêmes détecté.

Ces optimisations viseraient à réduire davantage le coût CPU sans perdre le gain en ratio.

5. Intégration dans un environnement applicatif :

Enfin, le projet pourrait être intégré dans un environnement applicatif plus large :

- **Compression réseau temps réel** : pour réduire la latence lors de transmissions de données.
- **Bases de données en mémoire** : où les colonnes d'entiers pourraient être stockées de façon compressée sans perte d'accès aléatoire.
- **Applications Big Data** : comme pré-traitement pour réduire les coûts de transfert entre serveurs.

Conclusion

Ce projet de Génie Logiciel avait pour objectif d'étudier, concevoir et comparer plusieurs méthodes de compression d'entiers basées sur le principe du Bit Packing, dans le but de réduire la taille des données transmises tout en conservant un accès direct et rapide à chaque élément. L'approche adoptée s'est inscrite dans une démarche expérimentale complète, combinant implémentation algorithmique, mesure des performances et analyse comparative.

Trois variantes ont été développées et évaluées :

- **BitPackingNoOverlap**, la version de base, simple et rapide, où chaque entier est stocké sans chevauchement ;
- **BitPackingOverlap**, une version plus compacte exploitant le chevauchement des mots binaires pour réduire le gaspillage de bits ;
- **BitPackingOverflow**, une approche avancée intégrant une zone de débordement pour traiter efficacement les valeurs extrêmes sans dégrader la compression globale.

Les résultats expérimentaux ont montré que :

- Pour des données homogènes, la méthode Overlap offre le meilleur ratio de compression, tandis que NoOverlap reste la plus rapide et la plus rentable en pratique.
- En présence de valeurs extrêmes peu fréquentes (outliers), Overflow devient la seule approche réellement efficace, maintenant un ratio de compression supérieur à 2× alors que les autres méthodes perdent toute efficacité.
- Le calcul du meilleur k et la mesure du temps total de transmission (T_{total}) ont permis de déterminer le seuil de latence à partir duquel chaque méthode devient avantageuse, reliant ainsi la compression aux conditions réelles de réseau.

Sur le plan logiciel, l'architecture retenue, basée sur une interface commune, une fabrique d'instances et une séparation claire des responsabilités, a permis de concevoir un code modulaire, extensible et facilement testable. Cette conception a facilité l'expérimentation, la comparaison entre méthodes et la reproductibilité des mesures.

Malgré ces réussites, plusieurs améliorations sont envisageables :

- La prise en charge des entiers négatifs, notamment via l'encodage ZigZag ou le décalage de plage,
- L'optimisation des performances grâce à la parallélisation ou à la vectorisation,
- L'extension du projet à d'autres types de données (flottants, booléens, données catégorielles).

En définitive, ce travail a permis de démontrer que le Bit Packing constitue une solution efficace, simple et flexible pour la compression sans perte de données numériques.

Il illustre l'importance du compromis entre temps de calcul et gain de transmission, ainsi que la nécessité d'adapter la méthode de compression au profil des données traitées. Le projet pose ainsi les bases d'un cadre expérimental réutilisable pour de futures études sur la compression adaptative et la transmission optimisée des données.